ICLR 2026 · Apple

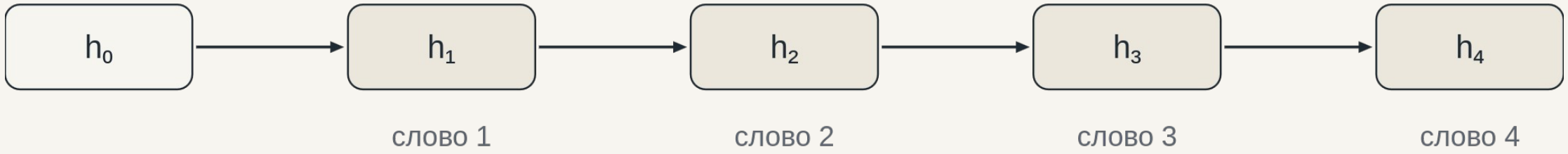
ParaRNN

Как посчитать нелинейную RNN сразу на всю последовательность

Danieli et al., 2025 · воспроизведение на RTX 3060 Laptop

RNN читает строго слева направо

Как человек с книгой: чтобы понять страницу 2, нужно помнить страницу 1.



Пока шаг 1 не посчитан, шаг 2 не начнётся. На тексте в тысячи слов это тысячи шагов подряд, а GPU, который умеет считать много сразу, почти простаивает.

Из-за этого RNN уступили место другим

Трансформер

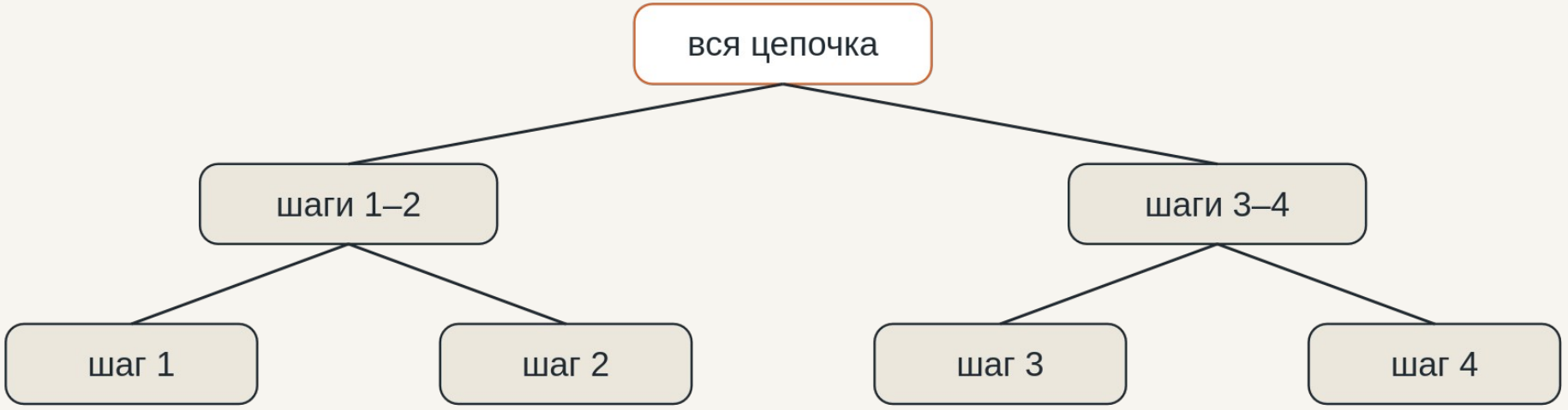
Считает всё сразу, поэтому быстро учится. Но на длинном тексте каждый новый токен смотрит на все прошлые, и это дорожает.

Mamba

Вернула рекуррентность, но линейную:
$h_t = a \cdot h_{prev} + b$.
Такие шаги можно склеивать парами.

Вопрос статьи: можно ли так же распараллелить нелинейную RNN?

Трюк Mamba: дерево вместо цепочки



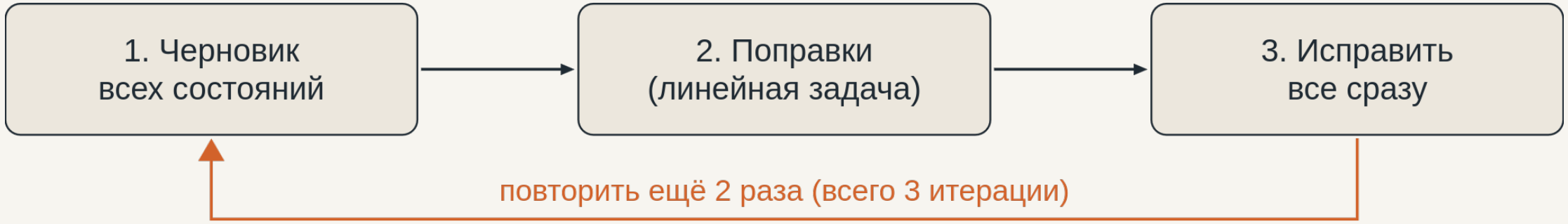
Соседние линейные шаги склеиваются в один, не дожидаясь начала текста. 4 шага — 2 уровня. 4096 шагов — 12 уровней вместо 4096.

Главная идея

Не решать шаги по очереди. Угадать все сразу и исправить все сразу.

Это метод Ньютона, применённый ко всей последовательности

Угадай и исправь



Черновик простой: каждый шаг считают так, будто прошлого нет. Как с эссе: сначала набросать целиком, потом править везде сразу.

Все шаги — одна система уравнений

Что нужно найти

$$h_1 = f(0, x_1)$$

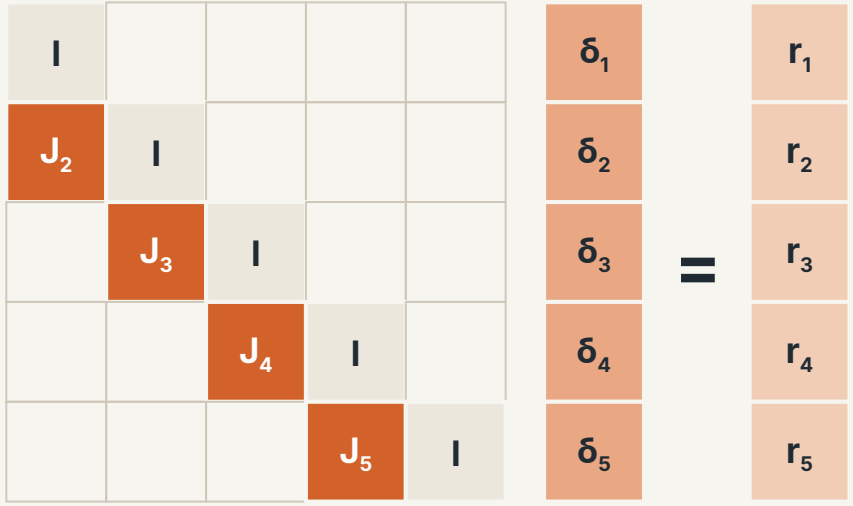
$$h_2 = f(h_1, x_2)$$

$$\dots$$

$$h_L = f(h_{prev}, x_L)$$

Неизвестны все h сразу, это L уравнений.
Метод Ньютона заменяет их линейными.

Один шаг Ньютона: линейная система



I — единичная матрица. J_t — якобиан шага t. **δ** — поправки, **r** — на сколько черновик промахнулся.

Каждое уравнение связано только с предыдущим, поэтому ненулевые блоки лежат на двух диагоналях. Такую систему можно решить быстро.

Каждая поправка — линейная задача

Вблизи черновика нелинейную ячейку заменяют касательной. Тогда поправка шага t зависит только от поправки шага prev:

$$\delta_t = A_t \cdot \delta_{prev} + r_t$$

A_t

насколько ячейка чувствительна к прошлому состоянию

r_t

насколько черновик промахнулся на этом шаге

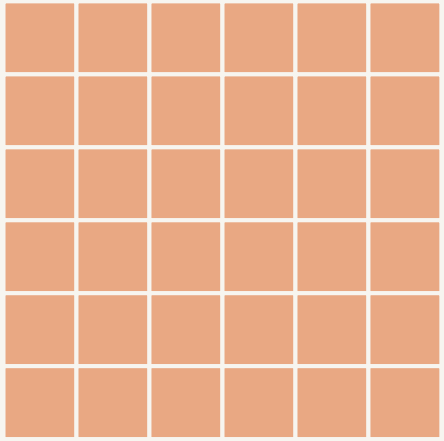
Это та же форма, что у Mamba, поэтому цепочку снова сворачивают деревом.

Что такое якобиан

Ячейка берёт старое состояние h и считает новое: $h_{next} = f(h_{prev}, x_t)$.

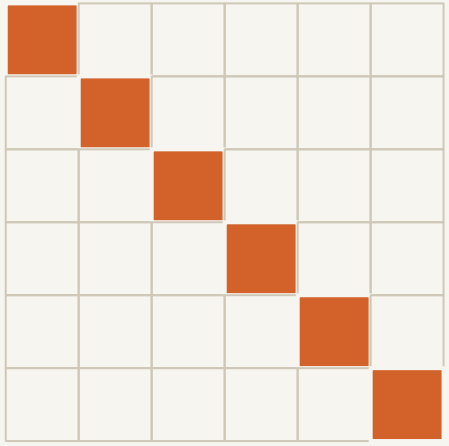
Якобиан J — таблица d × d. В строке i и столбце j стоит число: насколько сдвинется i-е число нового состояния, если чуть изменить j-е число старого.

Ньютону J нужен на каждом шаге: по нему он заменяет ячейку касательной.



плотный

d^2 чисел на шаг

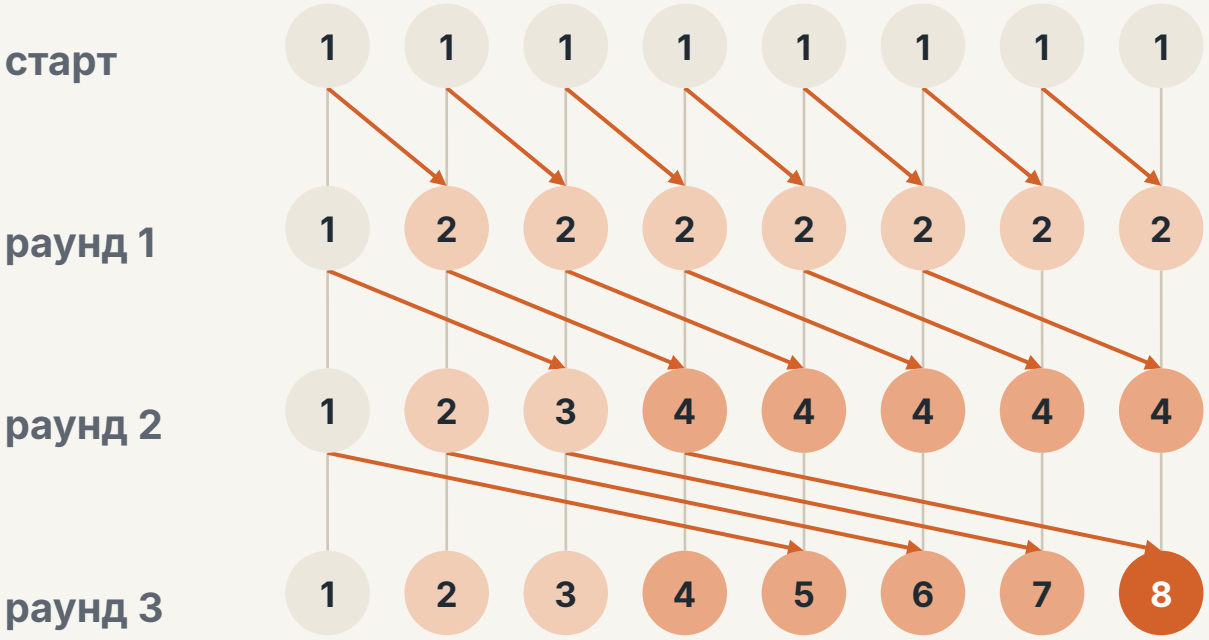


диагональный

d чисел на шаг

Если якобиан диагональный, вся «таблица» — один вектор: каждое число состояния зависит только от самого себя.

Параллельная редукция: 3 раунда вместо 8 шагов



Как читать схему

Число в кружке — сколько шагов уже склеено в этом месте.

Раунд 1: каждый берёт соседа слева.
Раунд 2: того, кто на 2 шага левее.
Раунд 3: на 4 шага.

Охват удваивается, поэтому для длины L хватает $\log_2 L$ раундов, а все кружки в раунде считаются одновременно.

Склейка двух кусков — снова шаг вида $\delta_t = A_t \cdot \delta_{prev} + r_t$, поэтому формула одна и та же на всех раундах.

Почему якобиан должен быть диагональным

≈ 17 ГБ

плотные якобианы

Память $L \cdot d^2$, одно перемножение двух якобианов стоит d^3 операций. Пример: L = 4096, d = 1024, fp32.

≈ 17 МБ

диагональные якобианы

Память $L \cdot d$, перемножение — d операций, всё считается независимо по каналам. Тот же пример, в 1000 раз меньше.

Так же устроены Mamba и S4D: диагональная матрица состояния. ParaRNN переносит этот приём на нелинейные ячейки.

Обратный проход: проще прямого

Прямому проходу нужен Ньютон, потому что ячейка нелинейна по состоянию. Градиенты же зависят от состояния линейно.

$$\nabla h_{prev} L = J_t^T \cdot \nabla h_{next} L + \partial L / \partial h_{prev}$$

Один проход, без Ньютона

Это та же линейная рекуррентность, поэтому годится тот же решатель редукции.

Идёт справа налево

Якобианы транспонируют, массив переворачивают и сдвигают. В коде это flip и roll.

Ячейки упростили, чтобы Ньютон был дешёвым

Полная матрица производных GRU — state × state на каждом токене. Хранить и перемножать её на длинном тексте дорого.

ParaGRU

Рекуррентные веса диагональные: каждое число состояния зависит только от себя. Производная — набор чисел, а не матрица.

ParaLSTM

Вход и забывание связаны. На каждый канал — маленький блок 2×2 вместо плотной матрицы.

Формулы GRU и LSTM те же. Меняется только то, какие матрицы плотные.

GRU: формулы и его якобиан

$$z = \sigma(A_z \cdot h_{prev} + B_z \cdot x + b_z)$$

насколько обновлять состояние

$$r = \sigma(A_r \cdot h_{prev} + B_r \cdot x + b_r)$$

сколько прошлого учитывать

$$c = \tanh(A_c \cdot (h_{prev} \odot r) + B_c \cdot x + b_c)$$

кандидат на новое состояние

$$h_{next} = (1 - z) \odot h_{prev} + z \odot c$$

смесь старого и кандидата

Три пути от h_{prev} к h_{next}

1. Напрямую: $(1 - z)$. Часть старого просто переходит дальше.

2. Через гейт z: $(c - h_{prev}) \cdot \sigma'(z) \cdot A_z$. Гейт сам зависит от h_{prev}.

3. Через кандидата: $z \cdot \sigma'(c) \cdot A_c \cdot (r + h_{prev} \cdot \sigma'(r) \cdot A_r)$. Здесь ещё и гейт r.

Если A диагональные, все три слагаемых — векторы, и J тоже вектор. В коде берут −J: система записана как h − f(h).

LSTM: состояние из двух частей

$$f = \sigma(A_f \cdot h_{prev} + B_f \cdot x + C_f \cdot c_{prev} + b_f)$$

забывание; ворота «подглядывают» в c_{prev}

$$c = f \odot c_{prev} + (1 - f) \odot z$$

новое c; z — кандидат

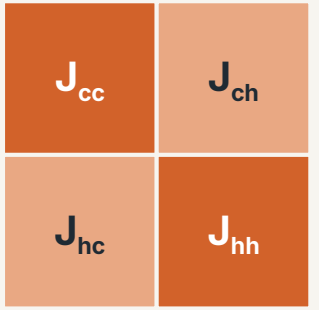
$$o = \sigma(A_o \cdot h_{prev} + B_o \cdot x + C_o \cdot c + b_o)$$

выходные ворота

$$h_{next} = o \odot \tanh(c)$$

выход ячейки

Якобиан — блок 2 × 2



Состояние — пара (c, h). Каждый блок сам диагональный: на каждый канал приходится своя матрица 2 × 2.

J_{cc}, J_{ch}: как прошлые c и h влияют на новое c.

J_{hc}, J_{hh}: h берётся из нового c, поэтому эти блоки строятся через J_{cc} и J_{ch}.

Диагональ: что теряем и чем платим

Что теряем

Внутри ячейки каналы не смешиваются: каждое число состояния видит только своё прошлое. Как в Mamba.

Как компенсируют

Смешивание делают матрицы входа B (блочные, по головам), свёртки и MLP между слоями.

Другой путь (Gonzalez и др., 2024): оставить полную ячейку и взять только диагональ её якобиана. Тогда Ньютон приближённый. ParaRNN меняет ячейку, и якобиан получается точным.

Идея статьи: не приближать якобиан, а подогнать сам модельный блок под дешёвый и точный якобиан.

Как этим пользоваться: три реализации

parallel

Только PyTorch. Для отладки и проб новых ячеек. Самый медленный из трёх.

parallel_CUDA

Решатель редукции на CUDA, якобианы собирает autograd. Градиенты считаются автоматически.

parallel_FUSED

Весь Ньютон в одном CUDA-ядре. Быстрее всех, но формулы якобианов нужно дать вручную.

Пользователь пишет только шаг ячейки f(h, x). Якобианы и параллельный решатель фреймворк собирает сам.

Что нужно для сходимости, и чего нет

Начальная догадка

$h^0_t = f(0, x_t)$: ячейка с нулевым прошлым. Считается параллельно, одним проходом.

Сколько итераций

Теория: до L, и тогда выигрыша нет. На практике 3–4: на старте обучения и после него.

Цена

Каждая итерация собирает якобианы и делает редукцию. Это лишняя работа, но глубина растёт как log L.

Для новой ячейки сходимость Ньютона нужно проверять отдельно: гарантии «за 3 шага» нет.

Как ячейку встраивают в языковую модель



Блок «свёртка → ячейка → гейт» повторяется N раз, это упрощённая схема. Свёртку и гейт авторы взяли из Mamba.

Обучали на одном корпусе, объём данных подбирали по правилу Chinchilla.

Берут Transformer DCLM, заменяют внимание на RNN-слой. Размеры: 400M, 1B, 2.9B и 7B параметров.

Что показала статья

до 665×

быстрее цикла

Полное параллельное применение ячейки, длина 512. Для двух ячеек: 447× и 665×.

35–37

токенов в секунду

Генерация у ParaGRU и ParaLSTM, у Mamba около 27. Не замедляется с длиной.

100%

на чётности

Нелинейные ячейки решают задачу. Mamba2 около 51%, Transformer 53%.

Данные авторов. k-hop: нелинейные ячейки 100%, Mamba2 98%, Transformer 78%. В MQAR все 100%.

Результат на моделях 7B

Числа из таблицы авторов: ячейку ставят вместо внимания и учат на SlimPajama. Это недели на нескольких H100.

модель	параметры	перплексия
Mamba2	6,96B	8,62
ParaLSTM	6,76B	9,16
ParaGRU	6,76B	9,19
Transformer	6,89B	9,55

Меньше перплексия — лучше. Вывод скромный: нелинейная RNN не проигрывает трансформеру того же размера.

Итог

Нелинейную RNN не нужно разворачивать циклом: три итерации Ньютона и дерево.

Нелинейность сохраняется, и именно она выучивает чётность.

Приложение: наше воспроизведение

Методика и выводы — в отчёте. Здесь только графики. Запуск: RTX 3060 Laptop, 6 ГБ.

Ньютон сходится за 3 итерации до точности float32.

Обучение в режимах цикла и параллельном совпадает.

Ускорение до 760× на длине 4096 (ParaGRU, fused).

Чётность: линейная RNN 0,50, ParaGRU и ParaLSTM 1,00.

Проверка 1: ответ совпадает с циклом

3

итерации Ньютона

хватает, чтобы совпасть с обычным циклом

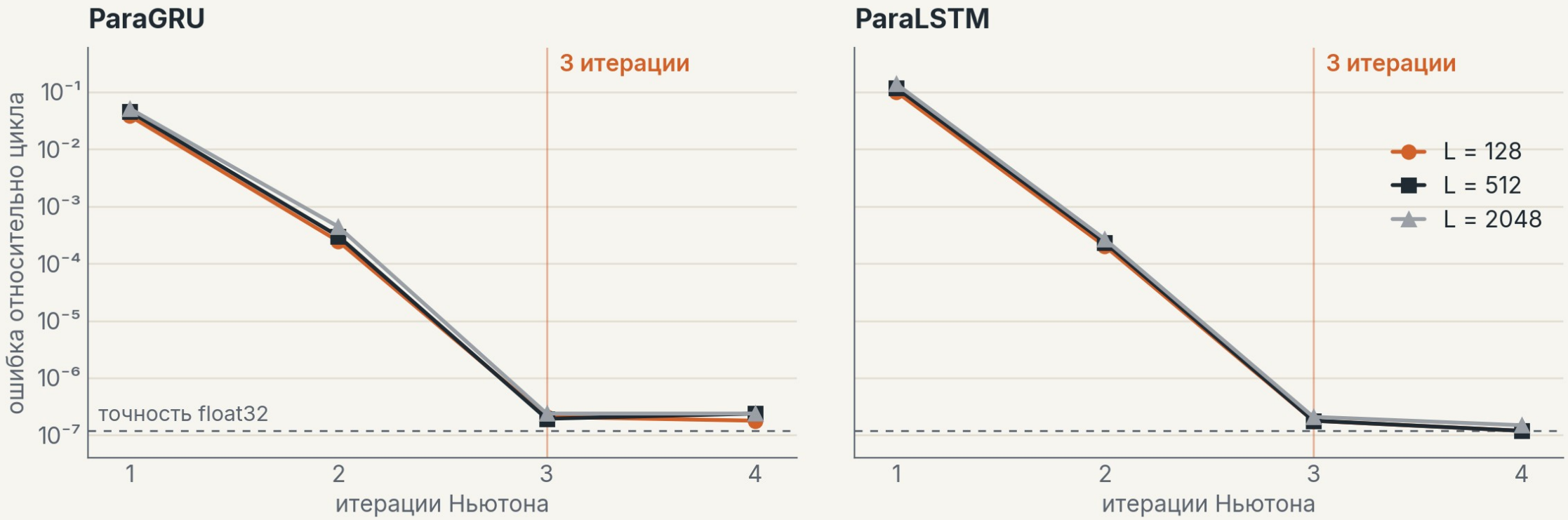
$\sim 10^{-7}$

расхождение

выход и градиенты, режимы PyTorch, CUDA и fused

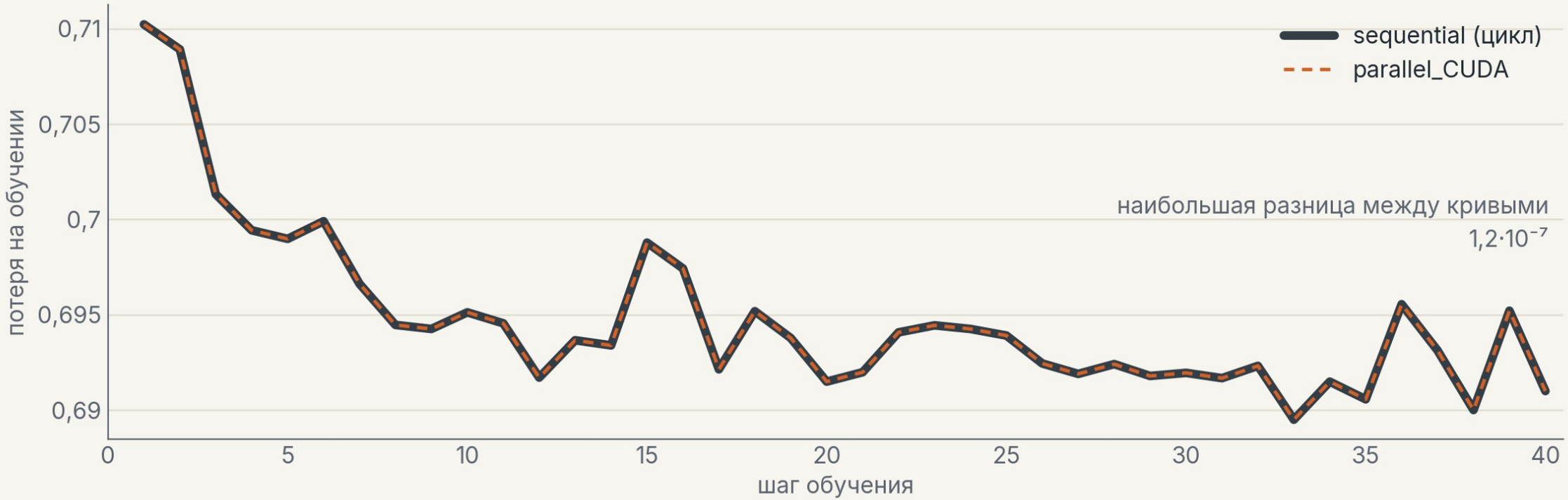
Длины 128, 512 и 2048 дают одну и ту же картину: длина почти не влияет на число итераций.

Ньютон сходится за три шага



Ошибка падает в 10 раз за шаг и после третьей итерации упирается в точность float32. Длины 128, 512 и 2048 ведут себя одинаково.

Обучение не отличается от цикла



Сорок шагов обучения ParaGRU в режимах sequential и parallel_CUDA. Кривые совпадают, максимальная разница $1{,}2 \cdot 10^{-7}$.

Проверка 2: скорость

≈760×

ParaGRU, длина 4096

цикл 2913 мс, fused 3,8 мс

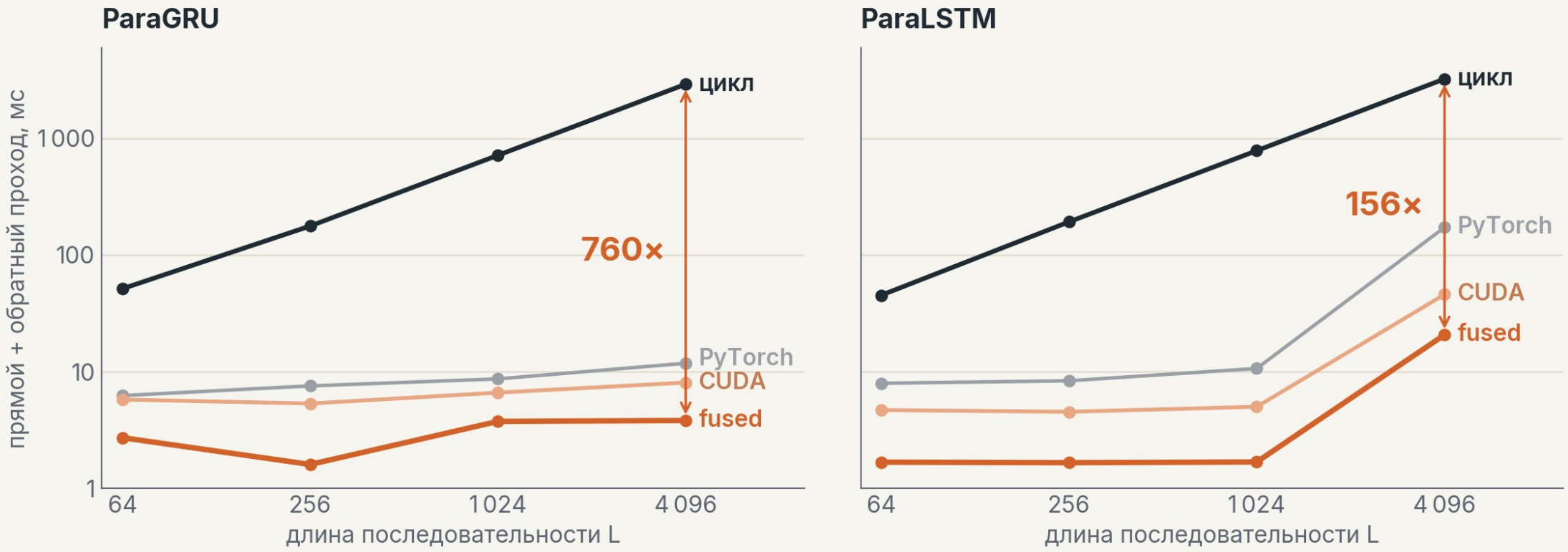
156×

ParaLSTM, длина 4096

блоки 2×2 тяжелее диагонали, поэтому выигрыш меньше

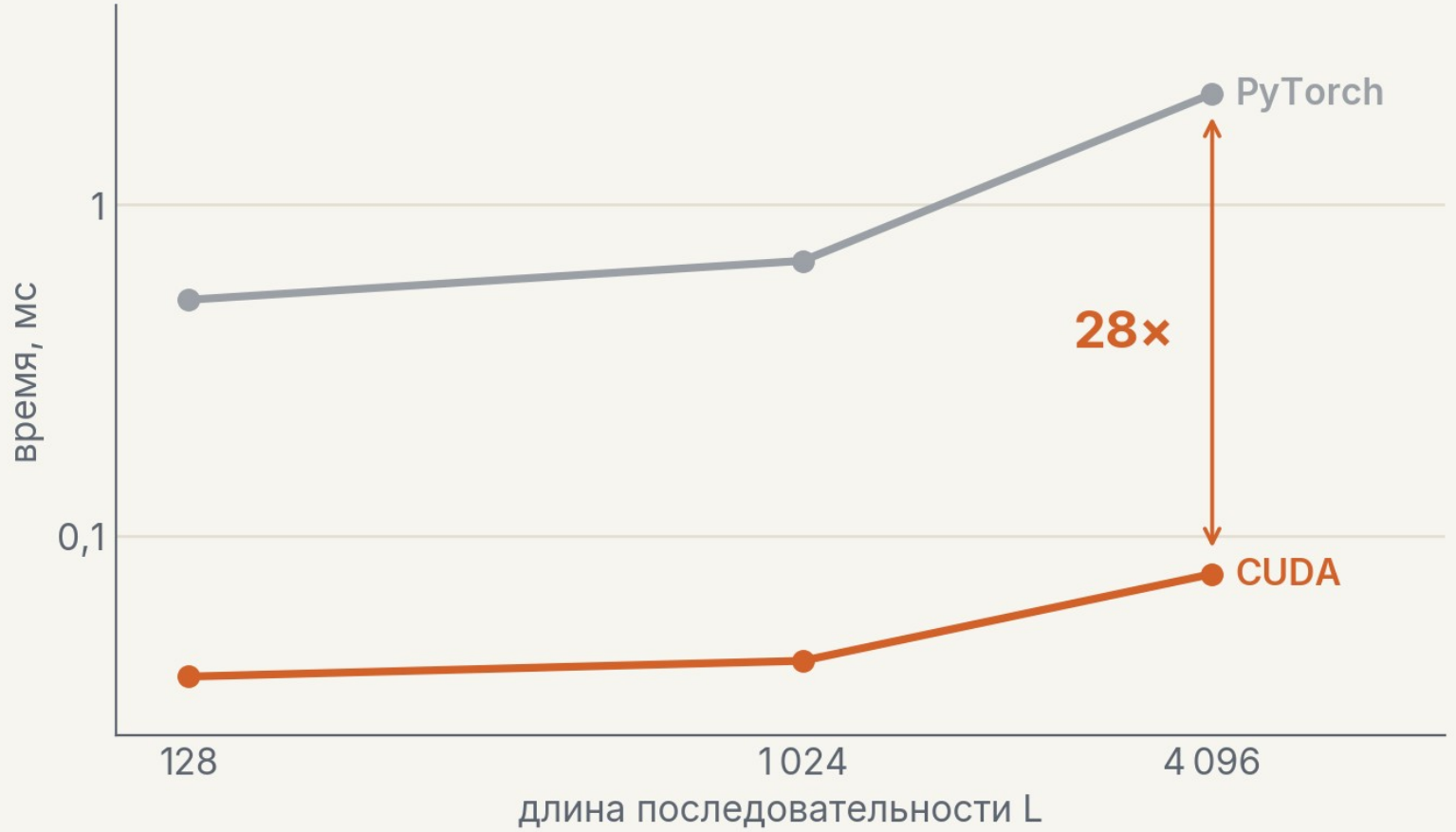
Время цикла растёт с длиной, fused почти нет. RTX 3060 Laptop, ширина 64, batch 2, прямой и обратный проход. В статье до 665× на A100 (только прямой проход): числа напрямую сравнивать нельзя, совпадает форма кривых.

Чем длиннее строка, тем больше выигрыш



Время прямого и обратного прохода. Цикл растёт почти линейно, параллельные режимы почти плоские. Логарифмическая шкала.

Где именно мы выигрываем



28×

ядро редукции

CUDA против PyTorch, длина 4096. Только решение линейной рекуррентности.

Проверка 3: зачем нужна нелинейность

Задача «чётность»: чётно ли число единиц в строке из нулей и единиц? Линейной рекуррентностью это выразить нельзя.

0,50

Линейная RNN

точность после 400 шагов

0,999

ParaGRU

точность, шаг 250

1,00

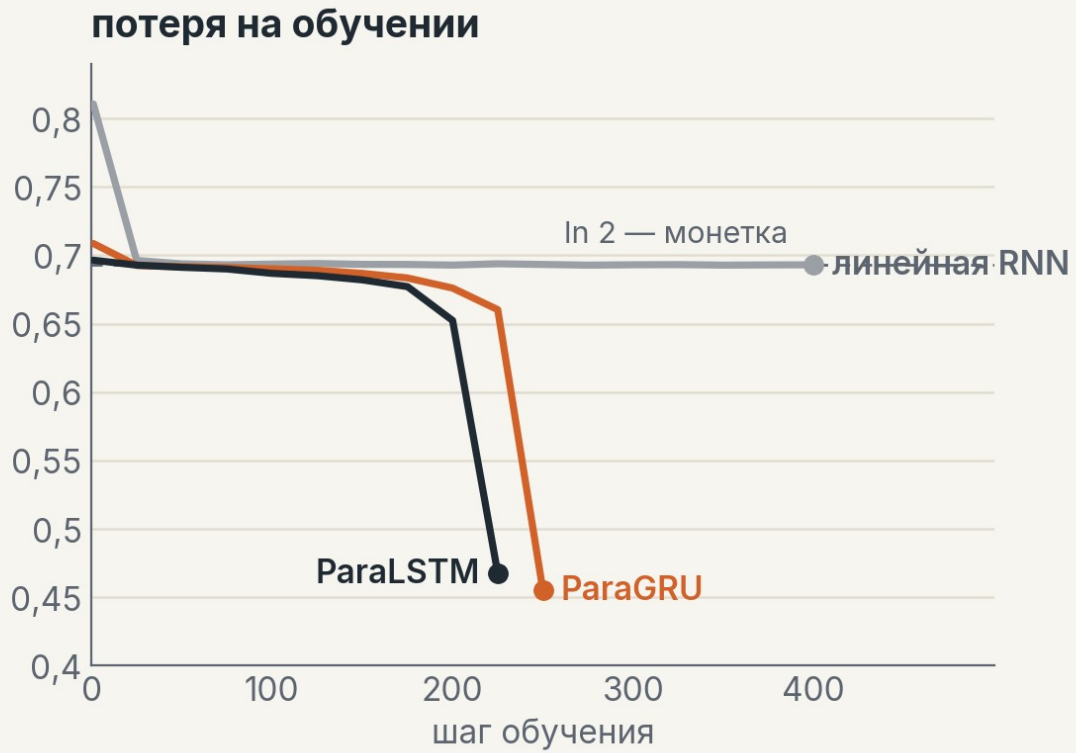
ParaLSTM

точность, шаг 225

Длина строки 50, один сид. В статье на этой задаче Mamba2 около 51%, трансформер 53%, нелинейные ячейки 100%.

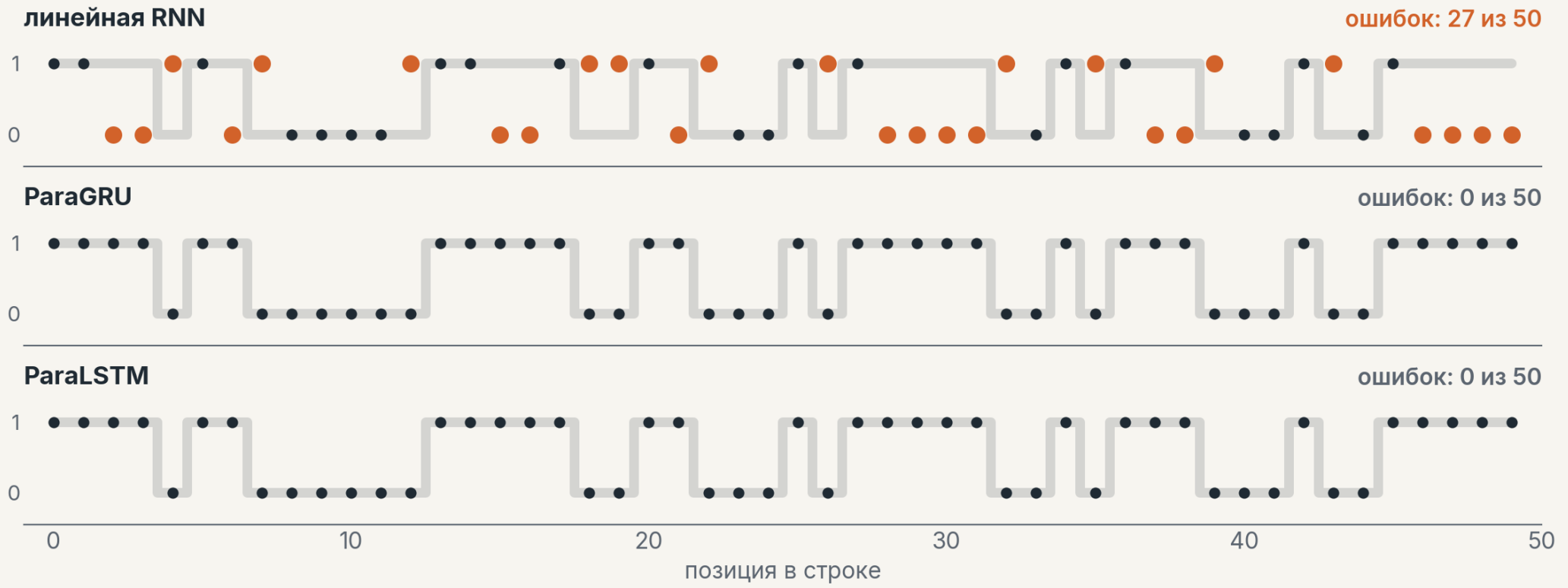
Обучение: линейная RNN стоит на месте





Линейная RNN остаётся на 0,50, как случайное угадывание. ParaGRU и ParaLSTM срываются вверх за 225–250 шагов.

Одна строка глазами моделей



Серая линия — правильный ответ, точки — предсказания. Оранжевые точки — ошибки: линейная ошибается в 27 из 50 позиций.

Точность по позиции в строке



Нелинейные ячейки верны на любой позиции, кроме первых двух. Линейная RNN держится около 0,5 везде.